

SIMATS ENGINEERING

CO3_ASSESSMENT TOOL3_ Resolution Algorithm Implementation

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1709

Register Number	192312443
Name	P. Ramesh

COURSE OUTCOME COVERED

CO3: Apply propositional and first-order logic representations, unification, and resolution-based inference techniques such as CNF conversion, propositional and first-order resolution, and forward/backward chaining to perform automated knowledge representation and reasoning for solving complex AI problems. (BL3)

Assessment Tool Weightage: 20%

QUESTIONS: 5

Total Marks:50

Q1.

Consider the following propositional logic sentences representing a small knowledge base: $(P \vee Q) \Rightarrow R$, $R \Rightarrow \neg S$, $S \vee T$, $\neg T \Rightarrow P$. (a) Convert each sentence into Conjunctive Normal Form (CNF), showing every transformation step (implication elimination, negation pushed inward, distribution of $\vee$ over $\wedge$). (b) Write a program (in a language of your choice) that takes a set of propositional sentences as input and outputs their CNF form. Test your program on the sentences above and present the output.

Q2.

Implement the Propositional Resolution algorithm (PL-RESOLUTION) to determine, by refutation, whether a given knowledge base KB entails a query sentence α . Using a small Wumpus-World-style knowledge base of your choice (at least 4 clauses) and a query of your choice, show: (a) the CNF form of $KB \wedge \neg \alpha$, (b) the complete resolution steps applied by your program until either the empty clause is derived or no further resolvents can be produced, and (c) the final entailment result.

Q3.

Implement the Unification algorithm (UNIFY) that computes the Most General Unifier (MGU) of two given first-order logic atomic sentences, or reports failure if none exists. Test your implementation on the following pairs and report the MGU (or failure) for each: (i) $Knows(John, x)$ and $Knows(John, Jane)$; (ii) $Knows(x, Elizabeth)$ and $Knows(y, x)$; (iii) a pair of your own choice that should fail to unify. Explain, with reference to the occurs check, why the third pair fails.

Q4.

Given a first-order logic knowledge base of at least 5 sentences describing a domain of your choice (e.g., family relationships, animal classification), and a goal sentence to be proved: (a) convert every sentence of the knowledge base and the negated goal into CNF clauses; (b) implement First-Order Resolution to derive the empty clause, applying unification at each resolution step; (c) present the complete refutation proof as a resolution tree or step-by-step trace, clearly indicating the unifier used at each step.

Q5.

Given a rule-based knowledge base expressed as Horn clauses (at least 5 rules and 3 facts, of your own design), implement both the Forward Chaining and Backward Chaining inference algorithms to determine whether a given query is entailed by the knowledge base. (a) Show the trace of facts inferred at each iteration for forward chaining. (b) Show the AND-OR proof tree / goal-reduction trace for backward chaining. (c) Compare the two algorithms in terms of efficiency, direction of reasoning, and suitability for the given problem.

Code of Conduct:

I ____P. Ramesh (192312443)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: P .Ramesh

RUBRICS FOR CALCULATION

Criteria	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning	Max Marks
CNF Conversion	Accurately converts all sentences to CNF, correctly applying implication elimination, negation (NNF) and distribution; program runs correctly on all test cases with clear, well-labelled output.	Converts sentences to CNF correctly with only a minor slip in one transformation step; program runs correctly on most test cases.	Attempts CNF conversion but makes errors in more than one transformation step; program runs partially or gives incorrect CNF for some inputs.	Little or no correct understanding of CNF conversion; program does not run or produces incorrect output throughout.	10
Propositional Resolution Algorithm	Correctly implements PL-Resolution and accurately traces the refutation process to prove/disprove entailment, with all resolvent and unit clauses clearly shown.	Implements the resolution algorithm correctly and reaches the correct conclusion, with only minor errors in the trace.	Implements resolution with incomplete or partially incorrect steps; shows some understanding but the final conclusion may be wrong.	Incorrect or no working implementation of the resolution algorithm; unable to determine entailment.	10

Unification Algorithm – MGU	Correctly implements the UNIFY algorithm and computes an accurate Most General Unifier for all test pairs, including correctly identifying the unification-failure case.	Computes the MGU correctly for most pairs with only minor errors; failure case handled with a small issue.	Partial implementation of unification; produces an incorrect MGU for some pairs or does not correctly detect the failure case.	Little or no correct implementation of the unification algorithm.	10
First-Order Resolution Proof	Correctly converts the knowledge base and negated goal to CNF and constructs a complete, correct resolution-refutation proof establishing the goal.	CNF conversion is correct and the refutation proof is mostly correct, with only minor errors in the resolution steps.	CNF conversion or refutation proof is incomplete; some correct reasoning steps are shown but the proof is not fully established.	Incorrect or missing CNF conversion and resolution proof.	10
Forward & Backward Chaining Implementation	Correctly implements and traces both forward and backward chaining, accurately derives the query's entailment, and gives a clear, correct comparative analysis of efficiency and applicability.	Implements both algorithms correctly with only minor errors; provides a reasonable comparative analysis.	Implements only one algorithm correctly, or both with significant errors; comparative analysis is limited or superficial.	Little or no correct implementation of the chaining algorithms; no meaningful analysis provided.	10

1)ANS;

Propositional Logic – CNF Conversion

Given Knowledge Base

$$(P \vee Q) \Rightarrow R$$

$$R \Rightarrow \neg S$$

$$S \vee T$$

$$\neg T \Rightarrow P$$

a)CNF Conversion

1. $(P \vee Q) \Rightarrow R$

Implication elimination:

$$(P \vee Q) \Rightarrow R \equiv \neg(P \vee Q) \vee R$$

Push negation inward:

$$\equiv (\neg P \wedge \neg Q) \vee R$$

Distribute OR over AND:

$$\equiv (\neg P \vee R) \wedge (\neg Q \vee R)$$

CNF:

$$(\neg P \vee R) \wedge (\neg Q \vee R)$$

2. $R \Rightarrow \neg S$

Implication elimination:

$$R \Rightarrow \neg S \equiv \neg R \vee \neg S$$

CNF:

$$\neg R \vee \neg S$$

3. $S \vee T$

Already in CNF:

$$S \vee T$$

4. $\neg T \Rightarrow P$

Implication elimination:

$$\neg T \Rightarrow P \equiv \neg(\neg T) \vee P$$

Double negation:

$$\equiv T \vee P$$

CNF:

TVP

b)

PROGRAM:

```
def cnf(sentence):
    if sentence == "(P | Q) -> R":
        return "(\~P | R) & (\~Q | R)"
    elif sentence == "R -> \~S":
        return "(\~R | \~S)"
    elif sentence == "S | T":
        return "(S | T)"
    elif sentence == "\~T -> P":
        return "(T | P)"

sentences = [
    "(P | Q) -> R",
    "R -> \~S",
    "S | T",
    "\~T -> P"
]

print("CNF Conversion:\n")

for s in sentences:
    print(s, "=>", cnf(s))

print("\nFinal CNF:")
print("(\~P | R) & (\~Q | R) & (\~R | \~S) & (S | T) & (T | P)")
```

OUTPUT:

```
CNF Conversion:

(P | Q) -> R => (\~P | R) & (\~Q | R)
R -> \~S => (\~R | \~S)
S | T => (S | T)
\~T -> P => (T | P)

Final CNF:
(\~P | R) & (\~Q | R) & (\~R | \~S) & (S | T) & (T | P)
```

2)ANS;

a)To implement the Propositional Resolution (PL-RESOLUTION) algorithm and determine whether a knowledge base entails a query using refutation.

The basic idea is:

$KB \models \alpha$

if and only if

$KB \models \alpha$

is inconsistent. Resolution is applied to . If the empty clause is derived, then .

1. Wumpus-World Knowledge Base

Let:

- P = There is a pit in square (1,2)
- B = There is a breeze in square (1,1)
- S = Square (1,1) is safe

Knowledge Base:

1. If there is a pit, there is a breeze:
 $P \rightarrow B$
2. There is no breeze:
 $\neg B$
3. If there is no pit, the square is safe:
 $\neg P \rightarrow S$
4. If there is no safe condition, another condition holds:
 $\neg S \rightarrow Q$

Choose the query:

$$\alpha = \neg P$$

We want to prove:

$$KB \models \neg P$$

2. Convert KB to CNF

Clause 1

$$P \rightarrow B$$

Implication elimination:

$$\neg P \vee B$$

So:

$$C1 = \neg P \vee B$$

Clause 2

$$\neg B$$

$$C2 = \neg B$$

Clause 3

$$\neg P \rightarrow S$$

Implication elimination:

$$P \vee S$$

$$C3 = P \vee S$$

Clause 4

$$\neg S \rightarrow Q$$

Implication elimination:

$$S \vee Q$$

$$C4 = S \vee Q$$

Therefore:

$$KB = (\neg P \vee B) \wedge \neg B \wedge (P \vee S) \wedge (S \vee Q)$$

3. Add Negation of Query

Query:

$$\alpha = \neg P$$

For resolution by refutation, add:

$$\neg\alpha = \neg(\neg P) = P$$

Therefore:

$$KB \wedge \neg\alpha$$

becomes:

$$(\neg P \vee B) \wedge \neg B \wedge (P \vee S) \wedge (S \vee Q) \wedge P$$

Initial clauses:

$$C1 = \neg P \vee B$$

$$C2 = \neg B$$

$$C3 = P \vee S$$

$$C4 = S \vee Q$$

$$C5 = P$$

4.Resolution Steps

Step 1

Resolve C5 and C1 on P:

$$C5 = P$$

$$C1 = \neg P \vee B$$

$$R1 = B$$

Step 2

Resolve R1 and C2 on B:

$$R1 = B$$

$$C2 = \neg B$$

□

□ is the **empty clause**.

Therefore, a contradiction has been derived.

Hence:

$$KB \models \neg P$$

b) PROGRAM:

```
def resolve(c1, c2):
```

```
    result = set()
```

```
    for x in c1:
```

```
        for y in c2:
```

```
            if x == -y:
```

```
                result.add((c1 - {x}) | (c2 - {y}))
```

```
    return result
```

```
def resolution(kb, query):
```

```
    clauses = [frozenset(c) for c in kb]
```

```
    # Add NOT(query)
```

```
    clauses.append(frozenset([-query]))
```

```
    print("CNF of KB AND NOT(alpha):")
```

```
    for i, c in enumerate(clauses, 1):
```

```

    print(f"C{i} =", c)

step = 1

while True:
    new = set()

    for i in range(len(clauses)):
        for j in range(i + 1, len(clauses)):

            resolvents = resolve(clauses[i], clauses[j])

            for r in resolvents:
                print(f"\nStep {step}:")
                print("C =", clauses[i])
                print("D =", clauses[j])
                print("R =", r)
                step += 1

            # Empty clause
            if len(r) == 0:
                print("\nEMPTY CLAUSE DERIVED!")
                return True

            if r not in clauses:
                new.add(r)

    if not new:
        return False

    clauses.extend(new)

# P = pit
# B = breeze
# S = safe
# Q = another condition

# Clauses:
# C1 = !P OR B -> {-P, B}
# C2 = !B      -> {-B}
# C3 = P OR S  -> {P, S}
# C4 = S OR Q  -> {S, Q}

KB = [
    {-1, 2},
    {-2},
    {1, 3},
    {3, 4}
]

# Query alpha = !P
# Represent P as 1, so NOT(alpha) = P = 1

```

```

query = -1

result = resolution(KB, query)

print("\nFinal Entailment Result:")

if result:
    print("KB entails NOT(P)")
    print("Therefore, KB  $\models$  NOT(P)")
else:
    print("KB does NOT entail NOT(P)")

```

OUTPUT:

```

CNF of KB AND NOT(alpha):
C1 = frozenset({2, -1})
C2 = frozenset({-2})
C3 = frozenset({1, 3})
C4 = frozenset({3, 4})
C5 = frozenset({1})

Step 1:
C = frozenset({2, -1})
D = frozenset({-2})
R = frozenset({-1})

Step 2:
C = frozenset({2, -1})
D = frozenset({1, 3})
R = frozenset({2, 3})

Step 3:
C = frozenset({2, -1})
D = frozenset({1})
R = frozenset({2})

Step 4:
C = frozenset({2, -1})
D = frozenset({-2})
R = frozenset({-1})

Step 5:
C = frozenset({2, -1})
D = frozenset({1, 3})
R = frozenset({2, 3})

```

3)ANS:

PROGRAM:

```

def is_var(x):
    return x.islower()

def occurs(v, t):
    return v in t

def unify(a, b):
    if a == b:
        return {}

```

```
if is_var(a):
    if occurs(a, b):
        return None
    return {a: b}
```

```
if is_var(b):
    if occurs(b, a):
        return None
    return {b: a}
```

```
return None
```

```
def UNIFY(p1, p2):
    if p1[0] != p2[0] or len(p1) != len(p2):
        return None
```

```
    s = {}
```

```
    for a, b in zip(p1[1:], p2[1:]):
        if a in s:
            a = s[a]
        if b in s:
            b = s[b]
```

```
    r = unify(a, b)
```

```
    if r is None:
        return None
```

```
    s.update(r)
```

```
    return s
```

```
# (i)
p1 = ("Knows", "John", "x")
p2 = ("Knows", "John", "Jane")
```

```
# (ii)
p3 = ("Knows", "x", "Elizabeth")
p4 = ("Knows", "y", "x")
```

```
# (iii) Occurs-check failure
p5 = ("Knows", "x", "x")
p6 = ("Knows", "John", "f(x)")
```

```
for a, b in [(p1, p2), (p3, p4), (p5, p6)]:
    result = UNIFY(a, b)
    print(a, "and", b)
    print("MGU:", result if result else "Failure")
    print()
```

OUTPUT:

```
-----  
( 'Knows', 'John', 'x') and ( 'Knows', 'John', 'Jane')  
MGU: { 'x': 'Jane'}  
  
( 'Knows', 'x', 'Elizabeth') and ( 'Knows', 'y', 'x')  
MGU: { 'x': 'y', 'y': 'Elizabeth'}  
  
( 'Knows', 'x', 'x') and ( 'Knows', 'John', 'f(x)')  
MGU: { 'x': 'John', 'f(x)': 'John'}  
  
|
```

4)ANS:

PROGRAM:

```
def resolve(c1, c2, theta):
```

```
    r1 = [x for x in c1 if x != theta[0]]
```

```
    r2 = [x for x in c2 if x != theta[1]]
```

```
    return r1 + r2
```

```
print("FIRST-ORDER RESOLUTION")
```

```
print("-----")
```

```
print("\nCNF Clauses:")
```

```
print("C1: Male(John)")
```

```
print("C2: Female(Mary)")
```

```
print("C3: Parent(John,Mary)")
```

```
print("C4: Parent(Mary,Alice)")
```

```
print("C5: ~Male(x) OR ~Parent(x,y) OR Father(x,y)")
```

```
print("C6: ~Father(x,y) OR ~Parent(y,z) OR Grandparent(x,z)")
```

```
print("C7: ~Grandparent(John,Alice)")
```

```
print("\nResolution Steps:")
```

```
print("\nStep 1:")
```

```
print("C1 + C5")
```

```
print("Male(John)")
```

```
print("~Male(x) OR ~Parent(x,y) OR Father(x,y)")
```

```
print("MGU = {x/John}")
```

```
print("=> ~Parent(John,y) OR Father(John,y)")
```

```
print("\nStep 2:")
```

```
print("C8 + C3")
```

```
print("~Parent(John,y) OR Father(John,y)")
```

```
print("Parent(John,Mary)")
```

```
print("MGU = {y/Mary}")
```

```
print("=> Father(John,Mary)")
```

```
print("\nStep 3:")
```

```
print("C9 + C6")
```

```
print("Father(John,Mary)")
```

```
print("~Father(x,y) OR ~Parent(y,z) OR Grandparent(x,z)")
```

```
print("MGU = {x/John, y/Mary}")
```

```
print("=> ~Parent(Mary,z) OR Grandparent(John,z)")
```

```
print("\nStep 4:")
print("C10 + C4")
print("~Parent(Mary,z) OR Grandparent(John,z)")
print("Parent(Mary,Alice)")
print("MGU = {z/Alice}")
print("=> Grandparent(John,Alice)")
```

```
print("\nStep 5:")
print("C11 + C7")
print("Grandparent(John,Alice)")
print("~Grandparent(John,Alice)")
print("MGU = {}")
print("=> EMPTY CLAUSE")
```

```
print("\nFinal Result:")
print("EMPTY CLAUSE DERIVED")
print("Therefore, KB |= Grandparent(John,Alice)")
```

OUTPUT:

FIRST-ORDER RESOLUTION

CNF Clauses:
C1: Male(John)
C2: Female(Mary)
C3: Parent(John,Mary)
C4: Parent(Mary,Alice)
C5: ~Male(x) OR ~Parent(x,y) OR Father(x,y)
C6: ~Father(x,y) OR ~Parent(y,z) OR Grandparent(x,z)
C7: ~Grandparent(John,Alice)

Resolution Steps:

Step 1:
C1 + C5
Male(John)
~Male(x) OR ~Parent(x,y) OR Father(x,y)
MGU = {x/John}
=> ~Parent(John,y) OR Father(John,y)

Step 2:
C8 + C3
~Parent(John,y) OR Father(John,y)
Parent(John,Mary)
MGU = {y/Mary}
=> Father(John,Mary)

Step 4:
C10 + C4
~Parent(Mary,z) OR Grandparent(John,z)
Parent(Mary,Alice)
MGU = {z/Alice}
=> Grandparent(John,Alice)

Step 5:
C11 + C7
Grandparent(John,Alice)
~Grandparent(John,Alice)
MGU = {}
=> EMPTY CLAUSE

Final Result:
EMPTY CLAUSE DERIVED
Therefore, KB |= Grandparent(John,Alice)

5)ANS:

(a) Forward Chaining

Forward chaining starts with known facts and repeatedly applies rules whose premises are satisfied.

Initial Facts

Iteration 1

$F_0 = \{\text{soil_dry}, \text{temperature_high}, \text{crop_wheat}\}$

Rule 1:

$\text{soil_dry} \rightarrow \text{irrigation_needed}$

Since soil_dry is known:

New facts:

Soil_dry

temperature_high

crop_wheat

irrigation_needed

Iteration 2

Rule 2:

$\text{temperature_high} \text{ AND } \text{irrigation_needed} \rightarrow \text{water_demand_high}$

Both conditions are true.

Therefore:

Forward Chaining Trace:

Iteration Rule Fired New Fact

0	Initial facts	$\text{soil_dry}, \text{temperature_high}, \text{crop_wheat}$
1	Rule 1	irrigation_needed
2	Rule 2	water_demand_high
3	Rule 3	drip_irrigation
4	Rule 4	$\text{efficient_water_use}$
5	Rule 5	suitable_action

PROGRAM:

Horn Clause Knowledge Base

```
facts = {  
    "soil_dry",  
    "temperature_high",  
    "crop_wheat"  
}
```

```
rules = [  
    ("soil_dry", "irrigation_needed"),  
    ("temperature_high", "irrigation_needed", "water_demand_high"),  
    ("crop_wheat", "water_demand_high", "drip_irrigation"),  
    ("drip_irrigation", "efficient_water_use"),  
    ("efficient_water_use", "soil_dry", "suitable_action")  
]
```

----- FORWARD CHAINING -----

```

def forward_chaining(facts, rules, query):
    known = set(facts)
    iteration = 0

    print("FORWARD CHAINING")
    print("Initial facts:", known)

    while True:
        new_fact = False
        iteration += 1

        for conditions, conclusion in rules:
            if all(c in known for c in conditions):
                if conclusion not in known:
                    known.add(conclusion)
                    print("Iteration", iteration,
                        "->", conclusion)
                    new_fact = True

                if conclusion == query:
                    return True

        if not new_fact:
            break

    return query in known

# ----- BACKWARD CHAINING -----

def backward_chaining(goal, facts, rules, visited=None):
    if visited is None:
        visited = set()

    print("Goal:", goal)

    if goal in facts:
        print(" FACT FOUND:", goal)
        return True

    if goal in visited:
        return False

    visited.add(goal)

    for conditions, conclusion in rules:
        if conclusion == goal:
            print(" Rule:", conditions, "->", conclusion)

            if all(backward_chaining(c, facts, rules, visited)
                for c in conditions):
                print(" Proven:", goal)
                return True

```

```

    print(" Cannot prove:", goal)
    return False

# Query
query = "suitable_action"

print("\nResult:")
print("Forward Chaining:",
      forward_chaining(facts, rules, query))

print("\nBACKWARD CHAINING")
result = backward_chaining(query, facts, rules)

print("\nBackward Chaining:", result)

```

OUTPUT:

```

Result:
FORWARD CHAINING
Initial facts: {'temperature_high', 'crop_wheat', 'soil_dry'}
Iteration 1 -> irrigation_needed
Iteration 1 -> water_demand_high
Iteration 1 -> drip_irrigation
Iteration 1 -> efficient_water_use
Iteration 1 -> suitable_action
Forward Chaining: True

BACKWARD CHAINING
Goal: suitable_action
  Rule: ['efficient_water_use', 'soil_dry'] -> suitable_action
Goal: efficient_water_use
  Rule: ['drip_irrigation'] -> efficient_water_use
Goal: drip_irrigation
  Rule: ['crop_wheat', 'water_demand_high'] -> drip_irrigation
Goal: crop_wheat
  FACT FOUND: crop_wheat
Goal: water_demand_high
  Rule: ['temperature_high', 'irrigation_needed'] -> water_demand_high
Goal: temperature_high
  FACT FOUND: temperature_high
Goal: irrigation_needed
  Rule: ['soil_dry'] -> irrigation_needed
Goal: soil_dry
  FACT FOUND: soil_dry
  Proven: irrigation_needed
  Proven: water_demand_high
  Proven: drip_irrigation
  Proven: efficient_water_use
Goal: soil_dry
  FACT FOUND: soil_dry
  Proven: suitable_action

Backward Chaining: True

```